

Chapter 13. Default methods

This chapter covers

- What default methods are
- Evolving APIs in a compatible way
- Usage patterns for default methods
- Resolution rules

Traditionally, a Java interface groups related methods together into a contract. Any (nonabstract) class that implements an interface *must* provide an implementation for each method defined by the interface or inherit the implementation from a superclass. But this requirement causes a problem when library designers need to update an interface to add a new method. Indeed, existing concrete classes (which may not be under the interface designers' control) need to be modified to reflect the new interface contract. This situation is particularly problematic because the Java 8 API introduces many new methods on existing interfaces, such as the `sort` method on the `List` interface that you used in previous chapters. Imagine all the angry maintainers of alternative collection frameworks such as Guava and Apache Commons who now need to modify all the classes implementing the `List` interface to provide an implementation for the `sort` method too!

But don't worry. Java 8 introduced a new mechanism to tackle this problem. It may sound surprising, but since Java 8 interfaces can declare methods with implementation code in two ways. First, Java 8 allowed *static methods* inside interfaces. Second, Java 8 introduced a new feature called *default methods* that allows you to provide a default implementation for methods in an interface. In other words, interfaces can now provide concrete implementation for methods. As a result, existing classes implementing an interface automatically inherit the default implementations if they don't provide one explicitly, which allows you to evolve interfaces nonintrusively. You've been using several default methods all along.

Other formats available



Audiobook



sort in the `List` interface and stream in

interface, which you saw in [chapter 1](#), is

new to Java 8 and is defined as follows:

```
default void sort(Comparator<? super E> c){
    Collections.sort(this, c);
}
```

```
}
```

Note the new `default` modifier before the return type. This modifier is how you can tell that a method is a default method. Here, the `sort` method calls the `Collections.sort` method to perform the sorting. Thanks to this new method, you can sort a list by calling the method directly:

```
List<Integer> numbers = Arrays.asList(3, 5, 1, 2, 6);  
numbers.sort(Comparator.naturalOrder());
```

1

- **1 `sort` is a default method in the `List` interface.**

Something else is new in this code. Notice that you call the `Comparator.naturalOrder` method. This new static method in the `Comparator` interface returns a `Comparator` object to sort the elements in natural order (the standard alphanumerical sort). The `stream` method in `Collection` you saw in [chapter 4](#) looks like this:

```
default Stream<E> stream() {  
    return StreamSupport.stream(spliterator(), false);  
}
```

Here, the `stream` method, which you used extensively in previous chapters to process collections, calls the `StreamSupport.stream` method to return a stream. Notice how the body of the `stream` method is calling the method `spliterator`, which is also a default method of the `Collection` interface.

Wow! Are interfaces like abstract classes now? Yes and no; there are fundamental differences, which we explain in this chapter. More important, why should you care about default methods? The main users of default methods are library designers. As we explain later, default methods were introduced to evolve libraries such as the Java API in a compatible way, as illustrated in [figure 13.1](#).

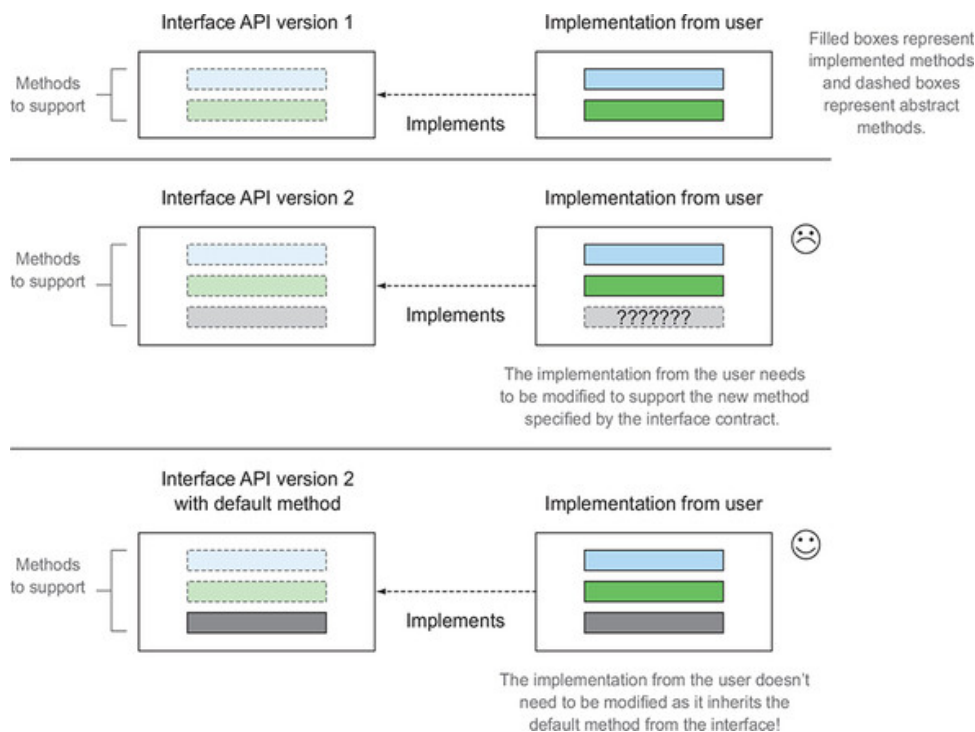
Figure 13.1. Adding a method to an interface

Other formats available



Audiobook





In a nutshell, adding a method to an interface is the source of many problems; existing classes implementing the interface need to be changed to provide an implementation for the method. If you're in control of the interface and all its implementations, the situation isn't too bad. But this is often not the case—and it provides the motivation for default methods, which let classes inherit a default implementation from an interface automatically.

If you're a library designer, this chapter is important because default methods provide a means of evolving interfaces without modifying existing implementations. Also, as we explain later in the chapter, default methods can help structure your programs by providing a flexible mechanism for multiple inheritance of behavior; a class can inherit default methods from several interfaces. Therefore, you may still be interested in finding out about default methods even if you're not a library designer.

Static methods and interfaces

A common pattern in Java is to define both an interface and a utility companion class defining many static methods for working with instances of the interface. `Collections` is a companion class to deal with

`Collection` objects, for example. Now that static methods can exist in-

Other formats available



Audiobook



uses in your code can go away, and their place inside an interface. These companion classes preserve backward compatibility.

The chapter is structured as follows. First, we walk you through a use case of evolving an API and the problems that can arise. Then we explain

what default methods are and discuss how you can use them to tackle the problems in the use case. Next, we show how you can create your own default methods to achieve a form of multiple inheritance in Java. We conclude with some more technical information about how the Java compiler resolves possible ambiguities when a class inherits several default methods with the same signature.

13.1. Evolving APIs

To understand why it's difficult to evolve an API when it's been published, suppose for the purpose of this section that you're the designer of a popular Java drawing library. Your library contains a `Resizable` interface that defines many methods that a simple resizable shape must support: `setHeight`, `setWidth`, `getHeight`, `getWidth`, and `setAbsoluteSize`. In addition, you provide several out-of-the-box implementations for it, such as `Square` and `Rectangle`. Because your library is so popular, you have some users who have created their own interesting implementations, such as `Ellipse`, using your `Resizable` interface.

A few months after releasing your API, you realize that `Resizable` is missing some features. It would be nice, for example, if the interface had a `setRelativeSize` method that takes as argument a growth factor to resize a shape. You might add the `setRelativeSize` method to `Resizable` and update your implementations of `Square` and `Rectangle`. But not so fast! What about all your users who created their own implementations of the `Resizable` interface? Unfortunately, you don't have access to and can't change their classes that implement `Resizable`. This problem is the same one that Java library designers face when they need to evolve the Java API. In the next section, we look in detail at an example that shows the consequences of modifying an interface that's already been published.

13.1.1. API version 1

The first version of your `Resizable` interface has the following methods:

```
public interface Resizable extends Drawable{
    int getWidth();
    int getHeight();
    void setWidth(int width);
    void setHeight(int height);
    void setAbsoluteSize(int width, int height);
}
```

Other formats available



User implementation

One of your most loyal users decides to create his own implementation of Resizable called Ellipse:

```
public class Ellipse implements Resizable {  
    ...  
}
```

He's created a game that processes different types of Resizable shapes (including his own Ellipse):

```
public class Game{  
    public static void main(String...args){  
        List<Resizable> resizableShapes =  
            Arrays.asList(new Square(), new Rectangle(), new Ellipse())  
        Utils.paint(resizableShapes);  
    }  
}  
  
public class Utils{  
    public static void paint(List<Resizable> l){  
        l.forEach(r -> {  
            r.setAbsoluteSize(42, 42);  
            r.draw();  
        });  
    }  
}
```

- **1 A list of shapes that are resizable**
- **2 Calling the setAbsoluteSize method on each shape**

13.1.2. API version 2

After your library has been in use for a few months, you receive many requests to update your implementations of Resizable: Square, Rectangle, and so on to support the setRelativeSize method. You come up with version 2 of your API, as shown here and illustrated in [figure 13.2](#):

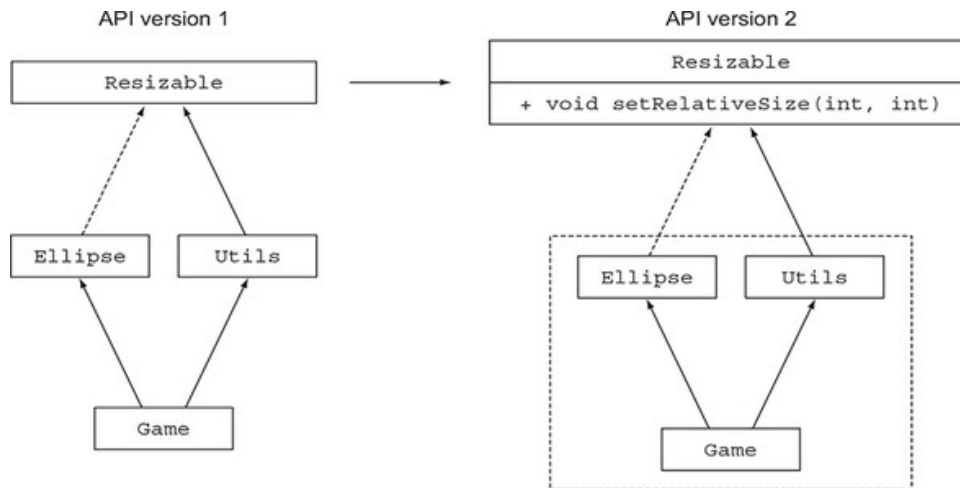
```
public interface Resizable {  
    int getWidth();  
    int getHeight();  
    void setAbsoluteSize(int width, int height);  
    void setRelativeSize(int width, int height);  
    void setRelativeSize(int wFactor, int hFactor);  
}
```

Other formats available



- **1 Adding a new method for API version 2**

Figure 13.2. Evolving an API by adding a method to Resizable.
Recompiling the application produces errors because it depends on the Resizable interface.



Problems for your users

This update of `Resizable` creates problems. First, the interface now demands an implementation of `setRelativeSize`, but the `Ellipse` implementation that your user created doesn't implement the method `setRelativeSize`. Adding a new method to an interface is *binary compatible*, which means that existing class file implementations still run without the implementation of the new method if no attempt is made to recompile them. In this case, the game will still run (unless it's recompiled) despite the addition of the `setRelativeSize` method to the `Resizable` interface. Nonetheless, the user could modify the method `Utils.paint` in his game to use the `setRelativeSize` method because the `paint` method expects a list of `Resizable` objects as an argument. If an `Ellipse` object is passed, an error is thrown at runtime because the `setRelativeSize` method isn't implemented:

```
Exception in thread "main" java.lang.AbstractMethodError:
    lambdasinaction.chap9.Ellipse.setRelativeSize(II)V
```

Second, if the user tries to rebuild his entire application (including `Ellipse`), he'll get the following compile error:

```
lambdasinaction/chap9/Ellipse.java:6: error: Ellipse is not abstract and
does not abstract method setRelativeSize(int,int) in Resizable
```

Other formats available

 Audiobook 

Evolution of an API creates backward incompatibilities, which is why evolving existing APIs, such as the official Java Collections API, causes problems for users of the APIs. You have alternatives to evolving an API, but they're poor choices. You could create a separate version of your API and maintain both the old and the new versions,

for example, but this option is inconvenient for several reasons. First, it's more complex for you to maintain as a library designer. Second, your users may have to use both versions of your API in the same code base, which affects memory space and loading time because more class files are required for their projects.

In this case, default methods come to the rescue. They let library designers evolve APIs without breaking existing code because classes that implement an updated interface automatically inherit a default implementation.

Different types of compatibilities: binary, source, and behavioral

There are three main kinds of compatibility when introducing a change to a Java program: binary, source, and behavioral compatibilities (see https://blogs.oracle.com/darcy/entry/kinds_of_compatibility). You saw that adding a method to an interface is binary compatible but results in a compiler error if the class implementing the interface is recompiled. It's good to know the different kinds of compatibilities, so in this sidebar, we examine them in detail.

Binary compatibility means that existing binaries running without errors continue to link (which involves verification, preparation, and resolution) without error after introducing a change. Adding a method to an interface is binary compatible, for example, because if it's not called, existing methods of the interface can still run without problems.

In its simplest form, *source compatibility* means that an existing program will still compile after introducing a change. Adding a method to an interface isn't source compatible; existing implementations won't recompile because they need to implement the new method.

Finally, *behavioral compatibility* means running a program after a change with the same input results in the same behavior. Adding a method to an interface is behavioral compatible because the method is never called in the program (or gets overridden by an implementation).

Other formats available

 **Audiobook** 

✕ nutshell

Adding a method to a published API disrupts existing implementations. *Default methods* are new in Java 8 to evolve APIs in a compatible way. Now an interface can contain method signatures for which an implementing class doesn't provide an implementation. Who imple-

ments them? The missing method bodies are given as part of the interface (hence, default implementations) rather than in the implementing class.

How do you recognize a default method? Simple: it starts with a default modifier and contains a body like a method declared in a class. In the context of a collection library, you could define an interface `Sized` with one abstract method `size` and a default method `isEmpty`, as follows:

```
public interface Sized {  
    int size();  
    default boolean isEmpty() {  
        return size() == 0;  
    }  
}
```

- **1 A default method**

Now any class that implements the `Sized` interface automatically inherits the implementation of `isEmpty`. Consequently, adding a method to an interface with a default implementation isn't a source incompatibility.

Now go back to the initial example of the Java drawing library and your game. Concretely, to evolve your library in a compatible way (which means that the users of your library don't have to modify all their classes that implement `Resizable`), use a default method and provide a default implementation for `setRelativeSize`, as follows:

```
default void setRelativeSize(int wFactor, int hFactor){  
    setAbsoluteSize(getWidth() / wFactor, getHeight() / hFactor);  
}
```

Because interfaces can now have methods with implementation, does that mean multiple inheritance has arrived in Java? What happens if an implementing class also defines the same method signature or default methods can be overridden? Don't worry about these issues for now; a few rules and mechanisms are available to help you deal with these issues. We explore them in detail in [section 13.4](#).

You may have guessed that default methods are used extensively in the

Other formats available



introduction of this chapter that the `Stream` interface that we used extensively in previous chapters. The `sort` method in the `List` interface is also a default method. Many of the functional interfaces we presented in [chapter 3](#)—such as `Predicate`, `Function`, and `Comparator`—also introduced new default methods, such as `Predicate.and` and

Function.andThen. (Remember that a functional interface contains only one abstract method; default methods are nonabstract methods.)

Abstract classes vs. interfaces in Java 8

What's the difference between an abstract class and an interface? Both can contain abstract methods and methods with a body.

First, a class can extend only *from one* abstract class, but a class can implement *multiple* interfaces.

Second, an abstract class can enforce a common state through instance variables (fields). An interface can't have instance variables.

To put your knowledge of default methods to use, have a go at quiz 13.1.

Quiz 13.1: removeIf

For this quiz, pretend that you're one of the masters of the Java language and API. You've received many requests for a `removeIf` method to use on `ArrayList`, `TreeSet`, `LinkedList`, and all other collections. The `removeIf` method should remove all elements from a collection that match a given predicate. Your task in this quiz is to figure out the best way to enhance the `Collections` API with this new method.

Answer:

What's the most disruptive way to enhance the `Collections` API? You could copy and paste the implementation of `removeIf` in each concrete class of the `Collections` API, but that solution would be a crime to the Java community. What else can you do? Well, all the `Collection` classes implement an interface called `java.util. Collection`. Great; can you add a method there? Yes. You've learned that default methods allow you to add implementations inside an interface in a source-compatible way. All classes that implement `Collection` (including classes from your users that aren't part of the `Collections` API) can use the implementa-

Other formats available



tion for `removeIf` is as follows (which is the official Java 8 `Collections` API). This so-

the `Collection` interface:

```
default boolean removeIf(Predicate<? super E> filter) {  
    boolean removed = false;  
    Iterator<E> each = iterator();
```

```

while(each.hasNext()) {
    if(filter.test(each.next())) {
        each.remove();
        removed = true;
    }
}
return removed;
}

```

13.3. Usage patterns for default methods

You've seen that default methods can be useful for evolving a library in a compatible way. Can you do anything else with them? You can create your own interfaces that have default methods too. You may want to do this for two use cases that we explore in the following sections: optional methods and multiple inheritance of behavior.

13.3.1. Optional methods

You're likely to have come across classes that implement an interface but leave empty some method implementations. Take the `Iterator` interface, for example, which defines `hasNext` and `next` but also the `remove` method. Before Java 8, `remove` was often ignored because users decided not to use that capability. As a result, many classes that implement `Iterator` have an empty implementation for `remove`, which results in unnecessary boilerplate code.

With default methods, you can provide a default implementation for such methods, so concrete classes don't need to explicitly provide an empty implementation. The `Iterator` interface in Java 8 provides a default implementation for `remove` as follows:

```

interface Iterator<T> {
    boolean hasNext();
    T next();
    default void remove() {
        throw new UnsupportedOperationException();
    }
}

```

Other formats available

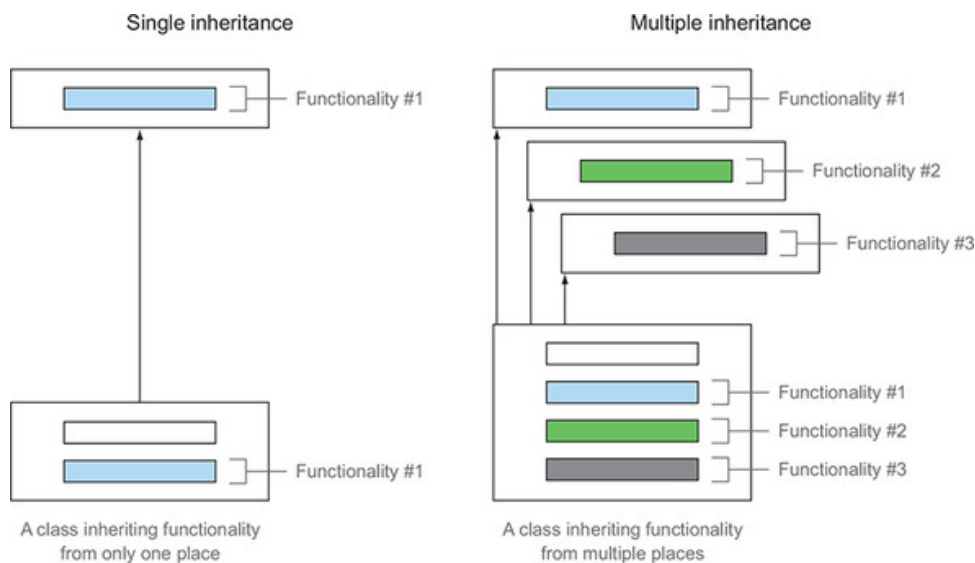
 Audiobook 

boilerplate code. Any class that implements the `Iterator` interface no longer needs to declare an empty `remove` method to ignore it because now it has a default implementation.

13.3.2. Multiple inheritance of behavior

Default methods enable something elegant that wasn't possible before: *multiple inheritance of behavior*, which is the ability of a class to reuse code from multiple places ([figure 13.3](#)).

Figure 13.3. Single inheritance versus multiple inheritance



Remember that classes in Java can inherit from only one other class, but classes have always been allowed to implement multiple interfaces. To confirm, here's how the class `ArrayList` is defined in the Java API:

```
public class ArrayList<E> extends AbstractList<E>      1
    implements List<E>, RandomAccess, Cloneable,
               Serializable {                          2
}                                                       }
```

- **1 Inherits from one class**
- **2 Implements four interfaces**

Multiple inheritance of types

Here, `ArrayList` is extending one class and directly implementing four interfaces. As a result, an `ArrayList` is a direct *subtype* of seven types: `AbstractList`, `List`, `RandomAccess`, `Cloneable`, `Serializable`, `Iterable`, and `Collection`. In a sense, you already have multiple inheritance of types.

Other formats available



Because interface methods can have implementations in Java 8, classes (and lambda expressions and method references) can inherit (or implement) methods (or implementation code) from multiple interfaces. In this example to show how you can use this capability, we'll use a set of minimal and orthogonal interfaces to let you achieve great reuse and composition of behavior inside your code base.

Minimal interfaces with orthogonal functionalities

Suppose that you need to define several shapes with different characteristics for the game you're creating. Some shapes should be resizable but not rotatable; some should be rotatable and movable but not resizable. How can you achieve great code reuse?

You can start by defining a stand-alone Rotatable interface with two abstract methods: `setRotationAngle` and `getRotationAngle`. The interface also declares a default `rotateBy` method that you can implement by using the `setRotationAngle` and `getRotationAngle` methods as follows:

```
public interface Rotatable {
    void setRotationAngle(int angleInDegrees);
    int getRotationAngle();
    default void rotateBy(int angleInDegrees){
        setRotationAngle((getRotationAngle () + angleInDegrees) % 360);
    }
}
```

- **1 A default implementation for the method rotateBy**

This technique is somewhat related to the template design pattern, in which a skeleton algorithm is defined in terms of other methods that need to be implemented.

Now any class that implements `Rotatable` will need to provide an implementation for `setRotationAngle` and `getRotationAngle` but will inherit the default implementation of `rotateBy` for free.

Similarly, you can define two interfaces that you saw earlier: `Moveable` and `Resizable`. Both interfaces contain default implementations. Here's the code for `Moveable`:

```
public interface Moveable {
    int getX();
    int getY();
    void setX(int x);
    void setY(int y);
    default void moveHorizontally(int distance){
        setX(getX() + distance);
    }
    default void moveVertically(int distance){
        setY(getY() + distance);
    }
}
```

Other formats available



And here's the code for `Resizable`:

```

public interface Resizable {
    int getWidth();
    int getHeight();
    void setWidth(int width);
    void setHeight(int height);
    void setAbsoluteSize(int width, int height);
    default void setRelativeSize(int wFactor, int hFactor){
        setAbsoluteSize(getWidth() / wFactor, getHeight() / hFactor);
    }
}

```

Composing interfaces

You can create different concrete classes for your game by composing these interfaces. Monsters, for example, can be moveable, rotatable, and resizable:

```

public class Monster implements Rotatable, Moveable, Resizable {
    ...
}

```

1

- **1 Needs to provide implementations for all abstract methods but not the default methods**

The Monster class automatically inherits the default methods from the Rotatable, Moveable, and Resizable interfaces. In this case, Monster inherits the implementations of rotateBy, moveHorizontally, moveVertically, and setRelativeSize.

Now you can call the different methods directly:

```

Monster m = new Monster();           1
m.rotateBy(180);                       2
m.moveVertically(10);                 3

```

- **1 Constructor internally sets the coordinates, height, width, and default angle.**
- **2 Calling rotateBy from Rotatable**
- **3 Calling moveVertically from Moveable**

Other formats available



declare another class that's moveable and as the sun. You don't need to copy and paste code; you can reuse the default implementations from the Moveable and Rotatable interfaces, as shown here.

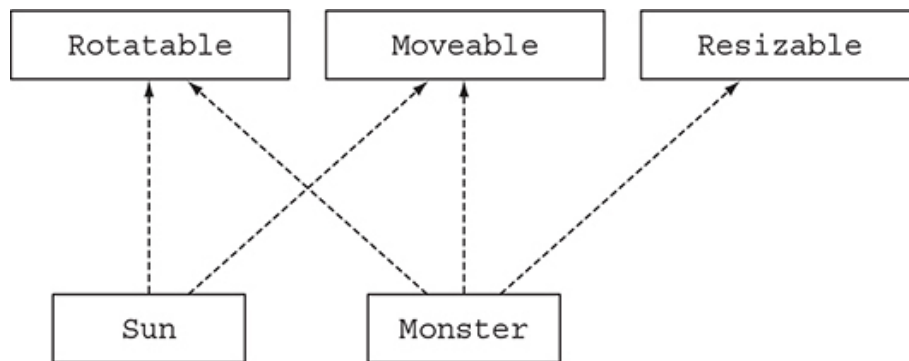
```
public class Sun implements Moveable, Rotatable {  
    ...  
}
```

1

- 1 Needs to provide implementations for all abstract methods but not the default methods

Figure 13.4 illustrates the UML diagram of this scenario.

Figure 13.4. Multiple behavior composition



Here’s another advantage of defining simple interfaces with default im-
plementations like the ones for your game. Suppose that you need to
modify the implementation of `moveVertically` to make it more efficient.
You can change its implementation directly in the `Moveable` interface,
and all classes implementing it automatically inherit the code (provided
that they didn’t implement the method themselves)!

Inheritance considered harmful

Inheritance shouldn’t be your answer to everything when it comes to
reusing code. For Inheriting from a class that has 100 methods and fields
to reuse one method is a bad idea, for example, because it adds unneces-
sary complexity. You’d be better off using *delegation*: create a method that
calls directly the method of the class you need via a member variable. For
this reason, you’ll sometimes find classes that are declared “final” inten-
tionally: they can’t be inherited from to prevent this kind of antipattern
or have their core behavior messed with. Note that sometimes, `final`
classes have a place. `String` is final, for example, because you don’t want

with such core functionality.

Other formats available

 Audiobook 

aces with default methods. By keeping
n achieve greater composition because

you can select only the implementations you need.

You've seen that default methods are useful for many usage patterns. But here's some food for thought: What if a class implements two interfaces that have the same default method signature? Which method is the class allowed to use? We explore this problem in the next section.

13.4. Resolution rules

As you know, in Java a class can extend only one parent class but implement multiple interfaces. With the introduction of default methods in Java 8, there's the possibility of a class inheriting more than one method with the same signature. Which version of the method should be used? Such conflicts probably are quite rare in practice, but when they do occur, there must be rules that specify how to deal with the conflict. This section explains how the Java compiler resolves such potential conflicts. We aim to answer questions such as "In the code that follows, which hello method is C calling?" Note that the examples that follow are intended to explore problematic scenarios; such scenarios won't necessarily happen frequently in practice:

```
public interface A {
    default void hello() {
        System.out.println("Hello from A");
    }
}

public interface B extends A {
    default void hello() {
        System.out.println("Hello from B");
    }
}

public class C implements B, A {
    public static void main(String... args) {
        new C().hello();
    }
}
```

- **1 What gets printed?**

In addition, you may have heard of the diamond problem in C++ in which a class can inherit two methods with the same signature. Which one gets chosen? Java 8 provides resolution rules to solve this issue too. Read on!

Other formats available



to know

when a class inherits a method with the same signature from multiple places (such as another class or interface):

1. Classes always win. A method declaration in the class or a superclass takes priority over any default method declaration.

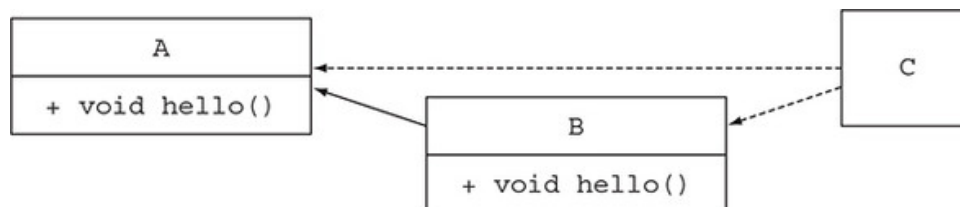
2. Otherwise, subinterfaces win: the method with the same signature in the most specific default-providing interface is selected. (If B extends A, B is more specific than A.)
3. Finally, if the choice is still ambiguous, the class inheriting from multiple interfaces has to explicitly select which default method implementation to use by overriding it and calling the desired method explicitly.

We promise that these are the only rules you need to know! In the next section, we look at some examples.

13.4.2. Most specific default-providing interface wins

Here, you revisit the example from the beginning of this section in which C implements both B and A, which define a default method called `hello`. In addition, B extends A. [Figure 13.5](#) provides a UML diagram for the scenario.

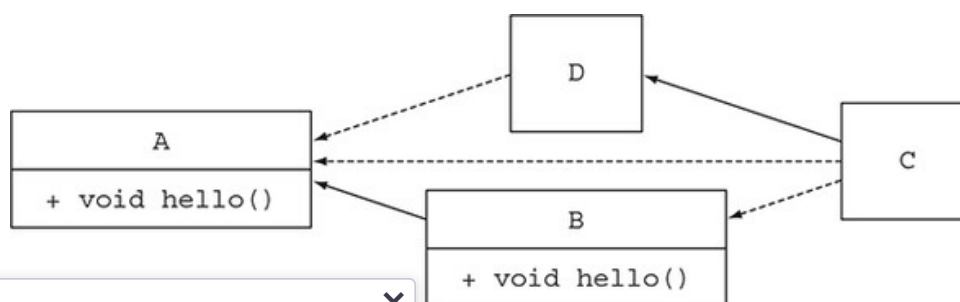
Figure 13.5. The most specific default-providing interface wins.



Which declaration of the `hello` method will the compiler use? Rule 2 says that the method with the most specific default-providing interface is selected. Because B is more specific than A, the `hello` from B is selected. Consequently, the program prints "Hello from B".

Now consider what would happen if C were inheriting from D as follows (illustrated in [figure 13.6](#)):

Figure 13.6. Inheriting from a class and implementing two interfaces



Other formats available



Audiobook



```

// ...
implements A{ }

public class C extends D implements B, A {
    // ...
    public static void main(String... args) {
        // ...
        new C().hello();
    }
}
  
```

```
}  
}
```

- **1 What gets printed?**

Rule 1 says that a method declaration in the class takes priority. But D doesn't override `hello`; it implements interface A. Consequently, it has a default method from interface A. Rule 2 says that if there are no methods in the class or superclass, the method with the most specific default-providing interface is selected. The compiler, therefore, has a choice between the `hello` method from interface A and the `hello` method from interface B. Because B is more specific, the program prints "Hello from B" again.

To check your understanding of the resolution rules, try quiz 13.2.

Quiz 13.2: Remember the resolution rules

For this quiz, reuse the preceding example, except that D explicitly overrides the `hello` method from A. What do you think will get printed?

```
public class D implements A{  
    void hello(){  
        System.out.println("Hello from D");  
    }  
}  
  
public class C extends D implements B, A {  
    public static void main(String... args) {  
        new C().hello();  
    }  
}
```

Answer:

The program prints "Hello from D" because a method declaration from a superclass has priority, as stated by rule 1.

Note that if D were declared as follows,

Other formats available



Audiobook



```
D implements A {  
    void hello();  
}
```

C would be forced to implement the method `hello` itself, even though default implementations exist elsewhere in the hierarchy.

13.4.3. Conflicts and explicit disambiguation

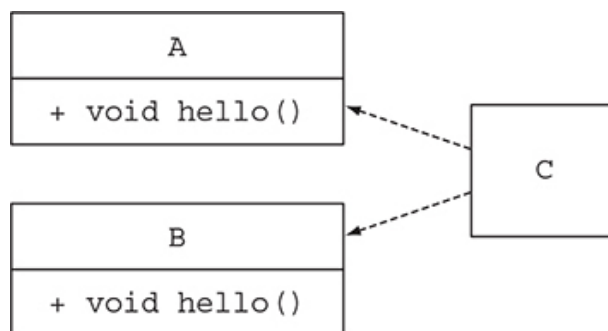
The examples you've seen so far could be resolved by the first two resolution rules. Now suppose that B doesn't extend A anymore (illustrated in [figure 13.7](#)):

```
public interface A {
    default void hello() {
        System.out.println("Hello from A");
    }
}

public interface B {
    default void hello() {
        System.out.println("Hello from B");
    }
}

public class C implements B, A { }
```

Figure 13.7. Implementing two interfaces



Rule 2 doesn't help you now because there's no more-specific interface to select. Both `hello` methods from A and B could be valid options. Thus, the Java compiler produces a compile error because it doesn't know which method is more suitable: "Error: class C inherits unrelated defaults for `hello()` from types B and A."

Resolving the conflict

There aren't many solutions to resolve the conflict between the two possible valid methods; you have to explicitly decide which method declaration you want C to use. To do so, you can override the `hello` method in class C and then, in its body, explicitly call the method you want to use.

Other formats available



✕ ax X. `super.m(...)` where X is the superclass you want to call. If you want C to use the default, the code looks like this:

```
public class C implements B, A {
    void hello(){
        B.super.hello();
    }
}
```

```
}  
}
```

- **1 Explicitly choosing to call the method from interface B**

Have a go at quiz 13.3 to investigate a related tricky case.

Quiz 13.3: Almost the same signature

For this quiz, assume that interfaces A and B are declared as follows:

```
public interface A{  
    default Number getNumber(){  
        return 10;  
    }  
}  
  
public interface B{  
    default Integer getNumber(){  
        return 42;  
    }  
}
```

Also assume that class C is declared as follows:

```
public class C implements B, A {  
    public static void main(String... args) {  
        System.out.println(new C().getNumber());  
    }  
}
```

What will the program print?

Answer:

C can't distinguish which method of A or B is more specific. For this reason, class C won't compile.

Other formats available



Audiobook



It sends shivers through the C++

community.

```
public interface A{  
    default void hello(){
```

```

        System.out.println("Hello from A");
    }
}
public interface B extends A { }
public interface C extends A { }
public class D implements B, C {
    public static void main(String... args) {
        new D().hello();
    }
}

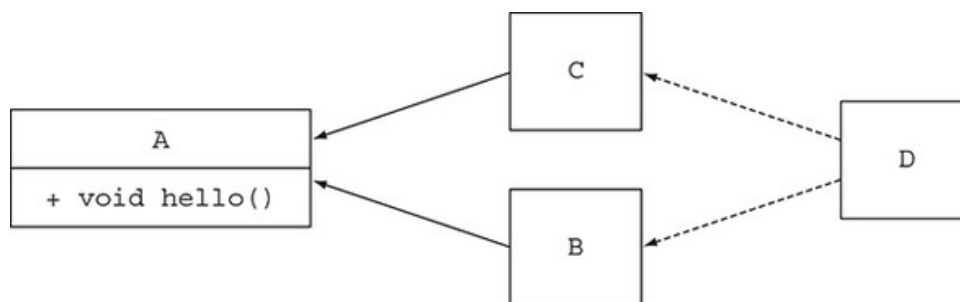
```

1

- **1 What gets printed?**

Figure 13.8 illustrates the UML diagram for this scenario. The problem is called a *diamond problem* because the diagram resembles a diamond. What default method declaration does D inherit: the one from B or the one from C? You have only one method declaration to choose. Only A declares a default method. Because the interface is a superinterface of D, the code prints "Hello from A".

Figure 13.8. The diamond problem



Now what happens if B also has a default hello method with the same signature? Rule 2 says that you select the most specific default-providing interface. Because B is more specific than A, the default method declaration from B is selected. If both B and C declare a hello method with the same signature, you have a conflict and need to solve it explicitly, as we showed earlier.

As a side note, you may wonder what happens if you add an abstract hello method (one that's not default) in interface C as follows (still no methods in A and B):

Other formats available



Audiobook

ends A {

The new abstract hello method in C takes priority over the default hello method from interface A because C is more specific. Therefore, class D

needs to provide an explicit implementation for `hello`; otherwise, the program won't compile.

C++ diamond problem

The diamond problem is more complicated in C++. First, C++ allows multiple inheritance of classes. By default, if a class D inherits from classes B and C, and classes B and C both inherit from A, class D has access to a copy of a B object and a copy of a C object. As a result, uses of methods from A have to be explicitly qualified: Are they coming from B or C? In addition, classes have state, so modifying member variables from B isn't reflected in the copy of the C object.

You've seen that the default method's resolution mechanism is simple if a class inherits from several methods with the same signature. Follow three rules systematically to solve all possible conflicts:

1. First, an explicit method declaration in the class or a superclass takes priority over any default method declaration.
2. Otherwise, the method with the same signature in the most specific default-providing interface is selected.
3. Finally, if there's still a conflict, you have to explicitly override the default methods and choose which one your class should use.

Summary

- Interfaces in Java 8 can have implementation code through default methods and static methods.
- Default methods start with a `default` keyword and contain a body, as class methods do.
- Adding an abstract method to a published interface is a source incompatibility.
- Default methods help library designers evolve APIs in a backward-compatible way.
- Default methods can be used for creating optional methods and multiple inheritance of behavior.

Other formats available



Audiobook



- Resolution rules exist to resolve conflicts when a class inherits from multiple methods with the same signature.
- A method declaration in a class or a superclass takes priority over default methods.
- Otherwise, the method with the same signature in the most specific default-providing interface is selected.
- When two methods are equally specific, a class must explicitly override this method, such as to select which one to call.

Other formats available



Audiobook

